Data and Information Quality Report

Project ID: 36

Assigned Dataset: 9

Students:
Marta Ravazzoni **11098105**
Alessandro Vitobello **10710941**

Project Structure

To maintain a modular and scalable workflow, we have decoupled each stage of the pipeline into dedicated Jupyter notebooks. The execution sequence and their respective outputs are detailed below:

Step	Notebook	Description	Output Format
1	diq_dataq.ipynb	Initial Data Quality Assessment	—
2	diq_datap.ipynb	YData Profiling	.html
3	diq_dataw.ipynb	Data Wrangling	.csv
4	diq_datac.ipynb	Data Clustering	.csv
5	diq_datam.ipynb	Missing Values Handling	.html, .csv
6	diq_datao.ipynb	Outlier Detection & Correction	.csv
7	diq_datad.ipynb	Duplicate Detection & Deduplication	.html, .csv
8	diq_dataq_final.ipynb	Final Data Quality Assessment	—

Setup Choices

Task	Libraries Used
Data Manipulation	pandas, numpy, re
Profiling & EDA	ydata-profiling, seaborn, matplotlib
Machine Learning	scikit-learn (IsolationForest, Ensemble), hdbscan
Dimensionality/NLP	umap-learn, sentence-transformers, scipy
Data Matching	recordlinkage

Quality Assessment

The initial phase involved a comprehensive database analysis to establish a baseline for data integrity. We extracted key metadata, including schema definitions and column names, while evaluating critical metrics such as duplicate counts, completeness ratios, and feature correlations (e.g., the relationship between *Superficie Altri Usi* and *Superficie Totale*).

```
# Show the total number of non-null values in the dataset
NOT_NULL = db.count().sum()
db.count().sum()
```

```
# Show the total number of null values in the dataset
NULL = db.isnull().sum().sum()
db.isnull().sum().sum()
```

```
# Calculate and display the completeness of the dataset
TOT = NULL + NOT_NULL
COMPLETENESS = NOT_NULL / TOT

COMPLETENESS = "{0: .1f}%".format(COMPLETENESS*100)
print(COMPLETENESS)
```

Data Profiling

We employed the ydata-profiling library to provide a comprehensive overview of the dataset, offering deep insights into data distribution, variability, and completeness.

```
# Generate the profiling report
PROFILE = ProfileReport(db, title="Profiling Report").to_file("db_REPORT_profiling.html")
```

Data Wrangling

This section marks the first stage of data transformation. To ensure consistency across the entire dataset, we began by standardizing all string values to lowercase.

```
# Convert all string values to lowercase
db = db.applymap(lambda x: x.lower() if isinstance(x, str) else x)
```

Following the initial cleaning, we focused on decomposing the *Settore Merceologico* field. By parsing the internal text strings, we successfully expanded this single attribute into five distinct columns to better categorize the data.

```
# Get unique values in 'Settore Merceologico' column
db['Settore Merceologico'].unique()
```

Consequently, the *Settore Merceologico* column was standardized to include only five valid categories: *Alimentare*, *Non Alimentare*, *Tabella Speciale Carburanti*, *Tabella Speciale Farmacie*, and *Tabella Speciale Monopolio*. Erroneous entries, such as 'Automobilli' or 'foglio 216 part. 83 sub. 716 (z.d. 8)' were excluded as they fall outside the attribute's defined domain.

Following the standardization of *Settore Merceologico*, we addressed the *Ubicazione* field. This attribute typically contains a composite of *Tipo Via*, *Via*, and *Civico*, alongside secondary details such as *Interno* and *Accesso*. To facilitate cleaning and extraction, we defined a 'valid triplet' consisting of the primary address components. We then initialized a secondary DataFrame (*db2*) from the source data (*db*) to isolate and clean the *Ubicazione* strings before further parsing.

```
# Create a copy of relevant columns
db2 = db[["Ubicazione", "Tipo Via", "Via", "Civico"]].copy(deep=True)
```

Subsequently, we derived a consolidated *Indirizzo* field by concatenating the validated triplet (*Tipo Via*, *Via*, and *Civico*). This new field served as a reference to validate the original *Ubicazione* entries; where a match could not be established, the *Ubicazione* value was flagged as “missing”.

```
# Check if Indirizzo is in Ubicazione
db2.loc[:, "Match Ubicazione"] = db2.apply(
    lambda r: r["Indirizzo"] in str(r["Ubicazione"]) if r["Indirizzo"] else False,
    axis=1
)

# Extract Resto Ubicazione
# Assuming Tipo Via, Via, and Civico from the original data are correct
db2.loc[:, "Resto Ubicazione"] = db2.apply(
    lambda r: (
        str(r["Ubicazione"]).replace(r["Indirizzo"], "").strip()
        if r["Match Ubicazione"]
        and isinstance(r["Ubicazione"], str)
        and isinstance(r["Indirizzo"], str)
        else np.nan
    ),
    axis=1
)
```

We then generated a *Resto Ubicazione* column within a temporary DataFrame (*tmp*) to store the residual strings from the original address field. Using this staging data, we parsed the remaining text to extract the *Isolato* and *Accesso* attributes.

```
# Categories
isolato_parts = [x for x in vals if x.startswith("isolato:")]
accesso_parts = [x for x in vals if x.startswith("accesso:")]
```

Finally, we isolated and extracted the *ZD* attribute from the remaining string data.

```
# z.d. parts
zd_parts = [x for x in vals if x.startswith("(z.d. ")]
```

Subsequently, we integrated the newly extracted attributes from the temporary DataFrame into *db2*. We decided to terminate further extraction from the *Resto Ubicazione* field, due to the high volume of missing values (exceeding 80%). We then consolidated our work by merging *db* with *db2*. The resulting schema is structured as follows:

	Alimentare	Non Alimentare	Tabella Speciale Carburanti	Tabella Speciale Farmacie	Tabella Speciale Monopolio	Insegna	Tipo Via	Via	Civico	Codice Via	ZD	Isolato	Accesso	Resto Ubicazione	Settore Storico Cf Preval	Superficie Vendita	Superficie Altri Usi	Superficie Tabelle Speciali	Superficie Totale
0	<NA>	<NA>	<NA>	<NA>	<NA>	NaN	CSO	ITALIA	34	526	1	NaN	NaN	NaN	speciale panificatori	0.0	NaN	NaN	0.0
1	<NA>	<NA>	<NA>	<NA>	<NA>	NaN	CSO	LODI	104	4068	4	NaN	NaN	NaN	NaN	16.0	5.0	NaN	21.0
2	<NA>	<NA>	<NA>	<NA>	<NA>	NaN	CSO	LODI	26	4068	5	NaN	NaN	NaN	speciale panificatori	0.0	NaN	NaN	0.0
3	<NA>	<NA>	<NA>	<NA>	<NA>	NaN	CSO	SEMPIONE	41	7137	8	NaN	NaN	NaN	NaN	120.0	NaN	NaN	120.0
4	<NA>	<NA>	<NA>	<NA>	<NA>	NaN	LGO	SCALABRINI GIOVANNI BATTISTA	2	5392	6	NaN	NaN	NaN	NaN	0.0	NaN	NaN	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
24131	True	False	False	False	True	NaN	RIP	DI PTA TICINESE	61	5150	6	071	esterno	NaN	cartoleria cancelleria	25.0	27.0	21.0	52.0
24132	True	True	False	False	True	i.p.	VIA	GALVANI LUIGI	34	1179	2	238	esterno	NaN	carburanti e lubrificanti	45.0	NaN	NaN	60.0
24133	False	True	False	False	True	tabacchi noli	VIA	MARINO TOMMASO	7	204	1	415	esterno	gll vittorio emanuele 000	articoli per fumatori	28.0	12.0	NaN	40.0
24134	False	True	False	False	True	NaN	VIA	ASELLI GASPARE	11	3167	3	NaN	NaN	NaN	commercio al dettaglio	7.0	125.0	5.0	132.0
24135	False	True	False	False	True	NaN	VLE	MONTE GRAPPA	2	1101	9	NaN	NaN	NaN	COMMERCIO AL DETTAGLIO	8.0	4.0	4.0	12.0

Clustering

In this section, we processed the *Settore Storico Cf Preval* attribute. As a preliminary step, we performed a string normalization by converting all characters to lowercase. Additionally, we handled null values by imputing them with the label "missing" to ensure the dataset was compatible with the subsequent clustering algorithm.

```
# Drop NaNs in the target column to prevent vectorization errors
df_clean = df.dropna(subset=[text_column]).copy()
```

We then employed UMAP (Uniform Manifold Approximation and Projection) for dimensionality reduction, facilitating more efficient clustering and a clearer visualization of the data distributions

```
# Dimensionality Reduction (UMAP)
umap_reducer = umap.UMAP(
    n_neighbors=15,
    n_components=15,
    min_dist=0.0,
    metric='cosine',
    random_state=42  #Fixed state for reproducibility
)
umap_embeddings = umap_reducer.fit_transform(embeddings)
```

With the dimensionality reduced, we proceeded to execute the clustering algorithm to identify distinct groupings within the processed feature space.

```
# Clustering (HDBSCAN)
clusterer = hdbscan.HDBSCAN(
    min_cluster_size=10,
    min_samples=5,
    metric='euclidean', #HDBSCAN usually runs on Euclidean distance of the reduced space
    cluster_selection_method='eom',
    prediction_data=True
)
cluster_labels = clusterer.fit_predict(umap_embeddings)
```

The clustering process successfully consolidated the *Settore Storico Cf Preval* attribute, reducing its cardinality from 2,119 unique labels to 230 distinct groupings.

Missing Values Handling

Missing values were handled through a combination of column removal, targeted imputation, and explicit placeholder assignment. First, variables exhibiting an excessive proportion (>65%) of missing observations, namely *Insegna* (71.0% of missing values), *Resto Ubicazione* (81.2% of missing values), and *Superficie Tabelle Speciali* (95.3% of missing values), were entirely removed from the dataset to avoid introducing noise or unreliable imputations.

```
db = db.drop(["Insegna"], axis = 1)
db = db.drop(["Resto Ubicazione"], axis = 1)
db = db.drop(["Superficie Tabelle Speciali"], axis = 1)
```

For surface-related measures, zero values in *Superficie Vendita* were explicitly recoded as missing, reflecting their semantic equivalence to absent information and enabling consistent treatment during imputation.

```
db['Superficie Vendita'] = db['Superficie Vendita'].replace(0,np.nan)
```

Missing values in a subset of key binary and numerical variables (*Alimentare*, *Non Alimentare*, *Tabella Speciale Carburanti*, *Tabella Speciale Farmacie*, *Tabella Speciale Monopolio*, *Superficie Vendita*, and *Superficie Altri Usi*) were then imputed using a Random Forest–based approach, that learns from the combined information of other relevant variables in the dataset to make the best possible estimates.

```
if feature in NUM:
  model = ensemble.RandomForestRegressor()
if feature in CAT:
  model = ensemble.RandomForestClassifier()
```

Following this step, the original variables with missing data were replaced by their imputed counterparts. No imputation techniques were applied to the remaining categorical variables (*Tipo Via*, *Via*, *Civico*, *Accesso*), as these features represent identifiers or labels that cannot be meaningfully inferred from observed values. Instead, they were completed by assigning an explicit "missing" category, preserving their categorical nature while avoiding row deletion. Similarly, selected numerical identifiers (*Isolato*, *ZD*, *Codice Via*) were filled with zero values, which were used solely as neutral placeholders rather than as statistically imputed estimates.

```
db["Tipo Via"] = db["Tipo Via"].fillna("missing")
db["Via"] = db["Via"].fillna("missing")
db["Civico"] = db["Civico"].fillna("missing")
db["Accesso"] = db["Accesso"].fillna("missing")
```

```
db["Isolato"] = db["Isolato"].fillna(0)
db["ZD"] = db["ZD"].fillna(0)
db["Codice Via"] = db["Codice Via"].fillna(0)
```

An exploratory consistency check showed that *Superficie Totale* was not consistently defined in the original dataset and could not be treated as an independently imputable variable. After removing *Superficie Tabelle Speciali* due to extensive missingness and imputing the remaining surface variables, *Superficie Totale* was recomputed as a derived measure as the row-wise sum of *Superficie Vendita* and *Superficie Altri Usi*, ensuring internal consistency after imputation.

```
db["Superficie Totale"] = db[
    ["Superficie Vendita", "Superficie Altri Usi"]
].sum(axis=1)
```

Outliers Detection and Correction

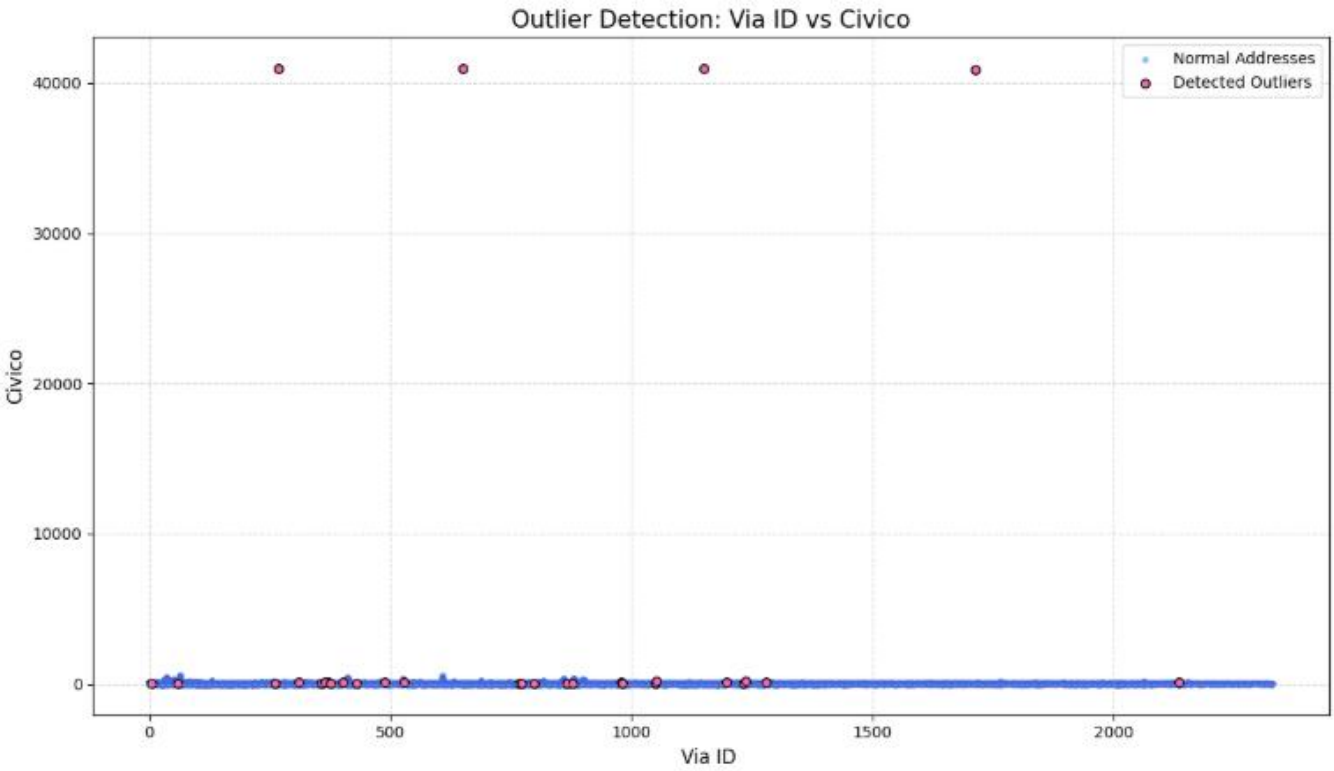
Outlier handling was conducted through a sequence of rule-based and model-based checks applied to key address and surface variables. For the street-number field (*Civico*), the pipeline first extracted the leading numeric component from the original string representation via a regular-expression rule and filled non-parsable cases with zero to enable quantitative screening; potential outliers were then flagged using a standard-deviation criterion, followed by a second, street-aware procedure that examined the ordered *Civico* progression within each street to identify anomalous jumps. The standard deviation threshold was intentionally set to 1 to increase the sensitivity of the outlier

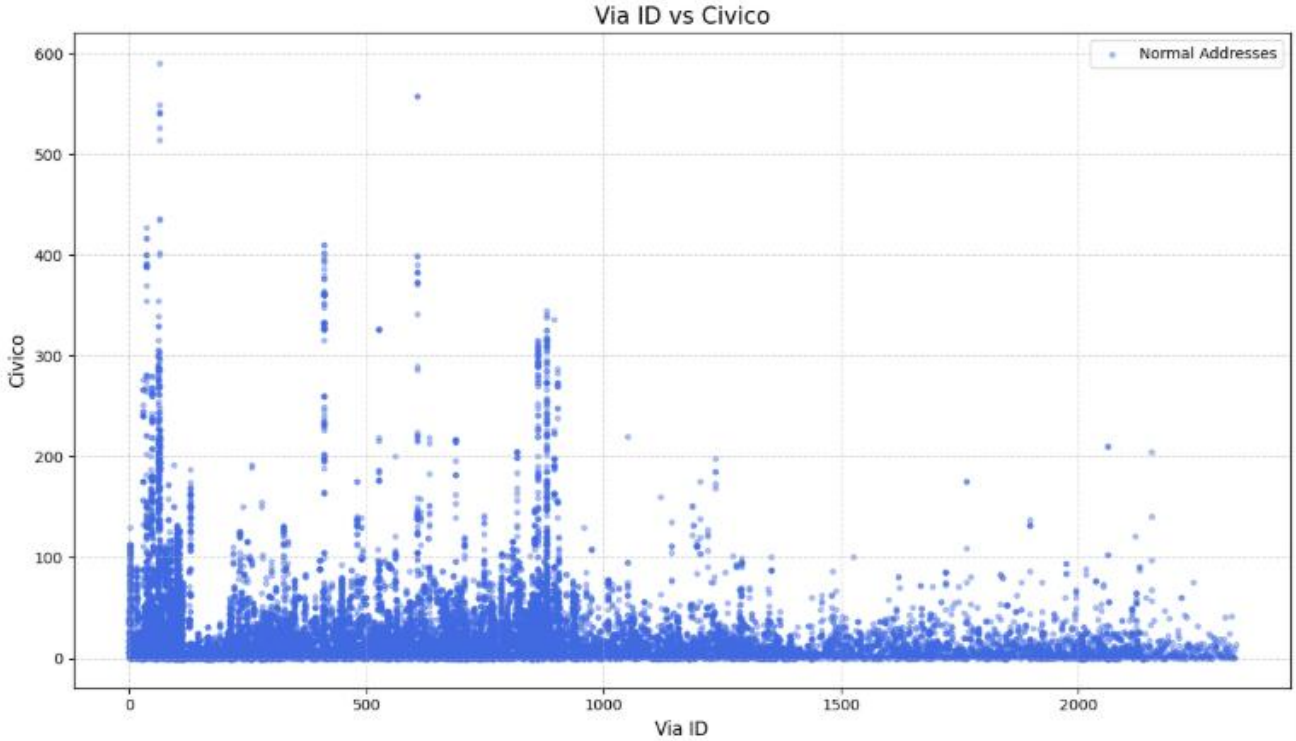
detection procedure. Given the non-Gaussian and discrete nature of the *Civico* variable, conventional thresholds (e.g. 2 or 3 standard deviations) were considered too conservative.

```
STD(df['Civico'], 1)
```

```
df['is_outlier'] = df.groupby('Via')['Civico'].transform(find_sequence_outliers)
```

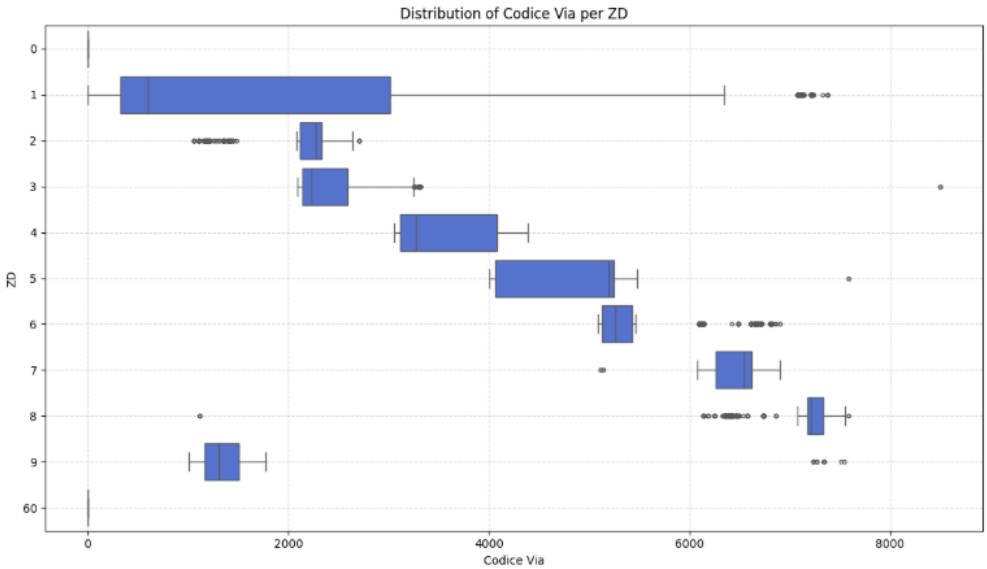
Records identified as *Civico* outliers were not removed; instead, their original *Civico* values were replaced with an explicit "missing" placeholder in the main dataset. Since these anomalous values were not intended to be used in the analysis, treating them as missing was equivalent to excluding their information while preserving the corresponding records. Diagnostic scatterplots of *Civico* versus an ordinal encoding of *Via* were generated before and after this correction to visually assess the impact of the procedure.





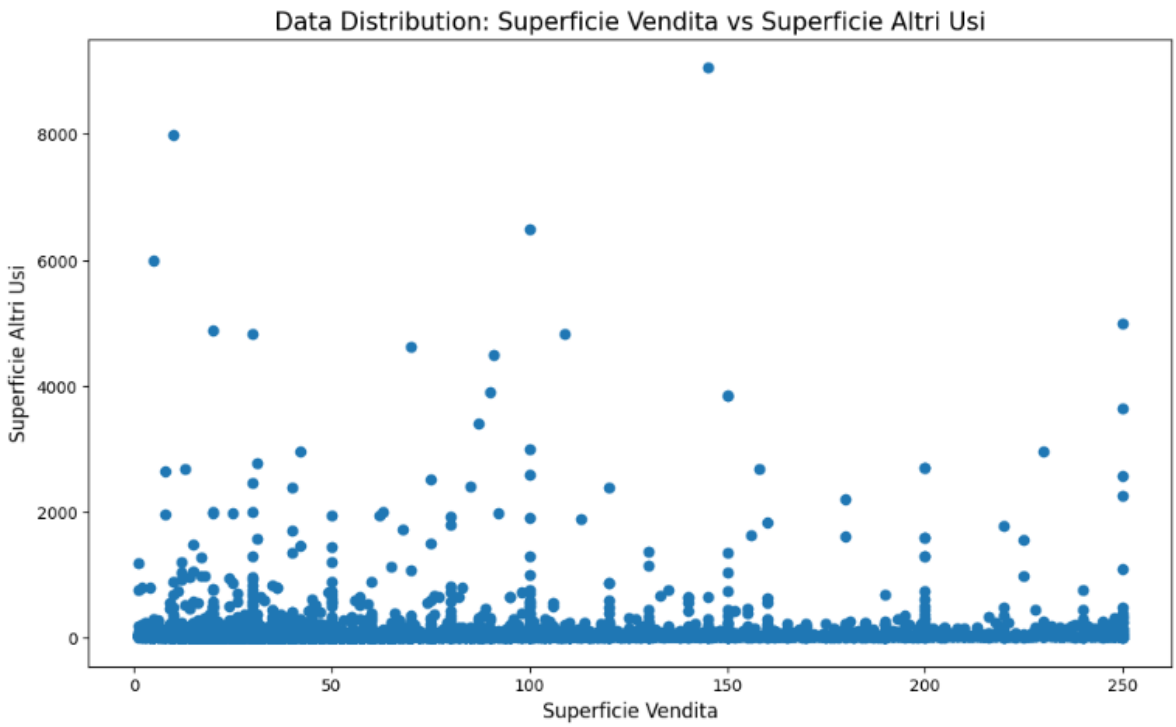
Outliers in *ZD* were addressed separately using a standard-deviation rule with a larger threshold (th = 3), replacing detected anomalies with zero. This choice was informed by preliminary exploratory analysis: several threshold values were tested, and th = 3 was selected because it identified 60 as the only outlier, consistent with prior inspection of the data, while preserving all other valid observations. A boxplot of *ZD* by *Codice Via*, together with inspection of unique *ZD* values, was used for exploratory assessment and validation of this choice.

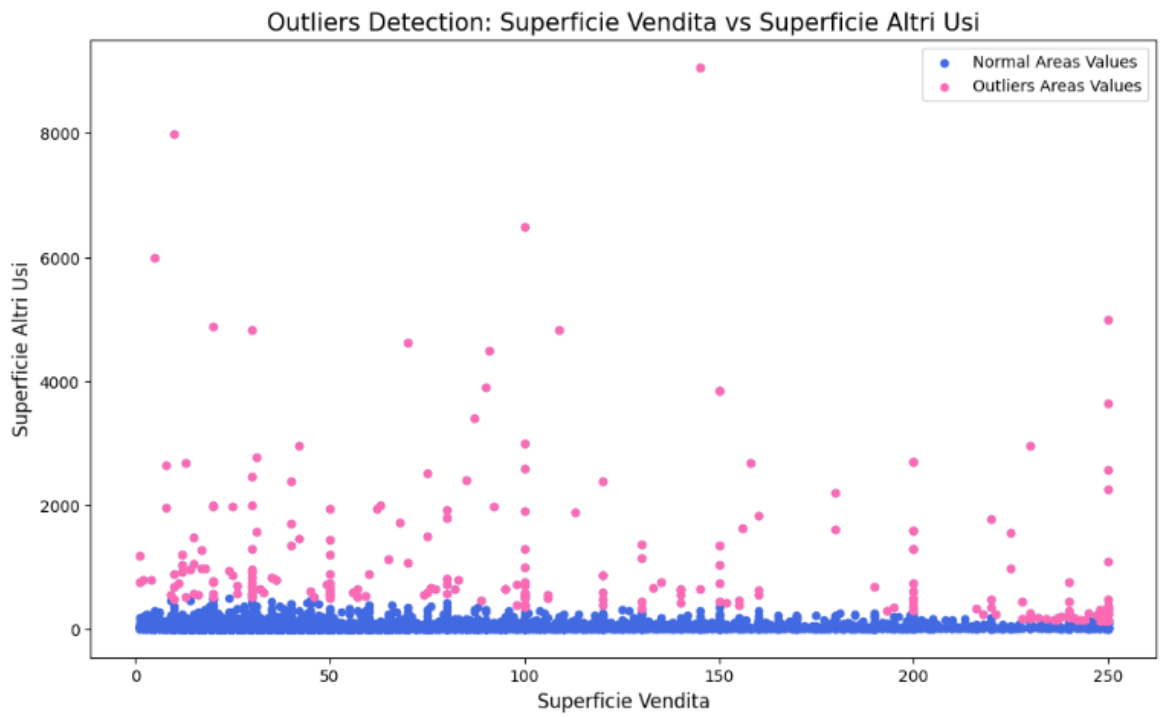
```
db['ZD'] = STD(db['ZD'], th=3)
```



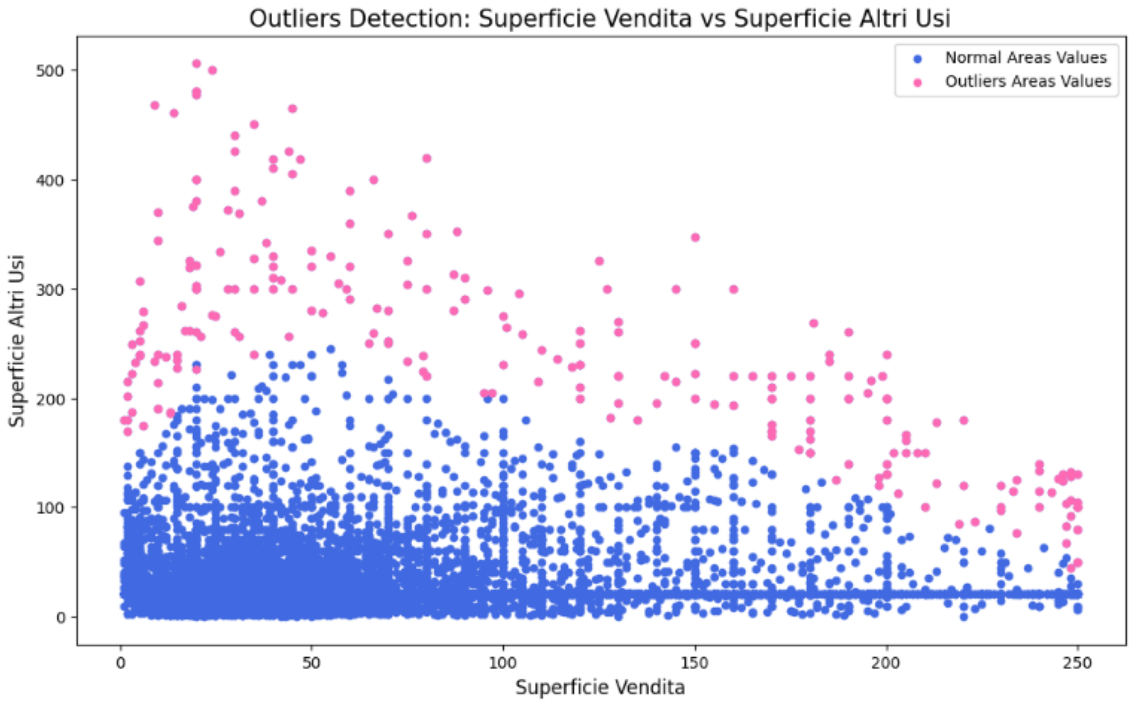
Finally, for the two continuous area measures (*Superficie Vendita* and *Superficie Altri Usi*), an Isolation Forest model was fitted to their joint distribution to identify multivariate outliers. The contamination level was set to 0.01 to reflect the expectation, based on prior exploratory analysis, that only a very small fraction of observations represented true joint anomalies, thereby ensuring a conservative detection of extreme cases.

```
model = IsolationForest(n_estimators=100, max_samples='auto', contamination=0.01, max_features=1.0)
```





The flagged observations had their area values set to missing (NaN) and were subsequently imputed using Random Forest models after categorical features were encoded, with the original area columns dropped and replaced by the imputed versions and the column order restored. The Isolation Forest procedure was then rerun and visualized on the corrected data to confirm the post-correction distribution.



Duplicates Detection and Data Deduplication

Duplicate handling was performed in two stages combining direct row-level checks with probabilistic record linkage. The pipeline first quantified exact duplicates and then removed strictly identical records.

```python
# Detect duplicates
duplicated = db.duplicated()
```

```python
# Remove duplicates
db = db.drop_duplicates()
```

To capture near-duplicates that may not be perfectly identical, a record-linkage procedure was subsequently applied. Candidate record pairs were generated by blocking on the *Via* field (i.e., comparisons were restricted to records sharing the same street, assuming that true duplicates were very unlikely to be on different streets).

```python
# Record linkage to find potential duplicates based on 'Via' column
indexer = recordlinkage.Index()
indexer.block('Via')
```

Each candidate pair was scored using a comparison object that included exact agreement checks on a set of columns alongside string-similarity comparisons using the Jaro–Winkler method for *Via* and *Settore Storico Cf Preval*. A Jaro–Winkler similarity threshold of 0.85 was adopted to allow for minor spelling variations in street names while maintaining a conservative matching criterion. This value was informed by exploratory inspection and chosen to minimize false positives.

```python
# Compare the candidate pairs
compare_cl = recordlinkage.Compare()
```

```python
# Columns that must match exactly
exact_cols = [
    'Alimentare',
    'Non Alimentare',
    'Tabella Speciale Carburanti',
    'Tabella Speciale Farmacie',
    'Tabella Speciale Monopolio',
    'Tipo Via',
    'Civico',
    'Codice Via',
    'ZD',
    'Isolato',
    'Accesso'
]
```

```python
for col in exact_cols:
    compare_cl.exact(col, col, label=col)
```

```python
# Fuzzy matching for 'Via' and 'Settore Storico Cf Preval' columns
compare_cl.string('Via', 'Via', method='jarowinkler', label='Via')
compare_cl.string('Settore Storico Cf Preval', 'Settore Storico Cf Preval', method='jarowinkler', threshold=0.85, label='Settore_Storico_Cf_Preval')
```

Pairs were classified as duplicates only when the computed feature vector indicated agreement on essentially all comparison features (implemented as a sum greater than 12, corresponding to all 13 features), and the corresponding duplicate records were removed from the dataset by dropping one member of each matched pair while preventing repeated deletions through index tracking.

```python
# Identify matches where all 13 features match
matches = features[features.sum(axis=1)>12]
```

```python
for i in matches.index:
    if i[1] not in indexes:
        DATA_DROP_DUPLICATES = DATA_DROP_DUPLICATES.drop([i[1]])
    indexes.append(i[1])
```

Final Observations

In conclusion, our data quality assessment demonstrates a significant improvement in the integrity and reliability of the dataset through targeted cleaning, imputation and deduplication processes.

Initially, the original dataset, comprised of 24,136 records, was flagged for containing numerous duplicate entries and missing values across several fields.

```
# Calculate and display the completeness of the dataset
TOT = NULL + NOT_NULL
COMPLETENESS = NOT_NULL / TOT

COMPLETENESS = "{0: .1f}%".format(COMPLETENESS*100)
print(COMPLETENESS)
```

```
82.1%
```

```
db[DUPLICATES].count().sum()
```

✓ 0.0s

```
330
```

By implementing a rigorous quality framework, we successfully refined the data into a final set of 23,454 unique records. This process achieved 100% completeness in the final version, effectively eliminating null values and redundant entries while ensuring that the remaining data points are both distinct and accurate.

```
# Calculate and display the completeness of the dataset
TOT = NULL + NOT_NULL
COMPLETENESS = NOT_NULL / TOT

COMPLETENESS = "{0: .1f}%".format(COMPLETENESS*100)
print(COMPLETENESS)
```

✓ 0.0s

```
100.0%
```

```
db[DUPLICATES].count().sum()
```

✓ 0.0s

```
0
```